M. H. SABOO SIDDIK COLLEGE OF ENGINEERING

8, Saboo Siddik Polytechnic road, Byculla, Mumbai-8

Department of Computer Science and Engineering (AI & ML)

DATABASE MANAGEMENT SYSTEM

S.E. – CSE (AIML)

SEM- IV

COURSE CODE : 2164112

Dr. Muhib A. Lambay
Asst. Professor

Module 2:
Relational Model
and
Relational Algebra

Relational Schema

- A relation schema describes the relation name (table name), and their attributes.
- It is a way to define a table.
- Collection of table definitions that gives the name of the table, lists the attributes and their data types, and identifies the primary key.
- Schema of a relation consists of
  - attribute definitions
    - name
    - type/domain
  - integrity constraints

Database

- A database consists of multiple relations.
- Information about an enterprise is broken up into parts, with each relation storing one part of the information.
- E.g. account : information about accounts
  depositor : which customer owns which account
  customer : information about customers
- Storing all information as a single relation such as
  bank(account_number, balance, customer_name, ..)
  results in:
  - repetition of information
  - the need for null values
- Normalization theory deals with how to design relational schemas

Relational Data Model

- Relational data model is the primary data model, which is used widely around the world for data storage and processing. This model is simple and have all the properties and capabilities required to process data with storage efficiency.
- Relational database tables and spreadsheets are not the same.
- Relational databases are more restrictive than Excel tables. The limits make them more powerful.
- Relational Model was proposed by E.F. Codd to model data in the form of relations or tables.

Relational Data Model

- In the relational model, all data is stored in relations (tables), and each relation consists of rows and columns. Each relation must have a header and body. The header is simply the list of columns in the relation. The body is the set of data that actually populates the relation, organized into rows.
- The Schema of a Relation is denoted by R(A1, A2, ...An)

  where, R is the name of the relation

  and the attributes of the relation are A1, A2, ..., An
- Example:

  CUSTOMER (Cust-id, Cust-name, Address, Phone#)

CUSTOMER is the relation name deefined over the four attributes: Cust-id, Cust-name, Address, Phone#

Relational Data Model

- Tables: In relation data model, relations are saved in the format of tables. A table has rows and columns, where rows represent records and columns represent the attributes.
- Tuple: A single row of a table, which contains a single record for that relation is called a tuple. A tuple is an ordered set of values generally represented within angle brackets. Order of tuples is irrelevant (tuples may be stored in an arbitrary order).
- Attribute: Columns in a table are referred as attributes.
- Degree: It refers to the number of attributes in the relation.
- Domain: The set of allowed values for each attribute is called the domain of the attribute. A pool of values from which the actual values appearing in a given column are drawn. Attribute values are required to be atomic; that is, indivisible. The special value null is a member of every domain indicating that the value is "unknown" or inapplicable to certain tuples. Example: "mobile_number" is a set of 10 digits.

Relational Data Model

- Relation Instance/Relation State: A finite set of tuples in the relational database system represents relation instance. Relation instances do not have duplicate tuples.
- A relation instance(r) is a collection of tuples for a given relational schema at an exact point of time.
- Denoted By: r = {t1,t2,.....tm}

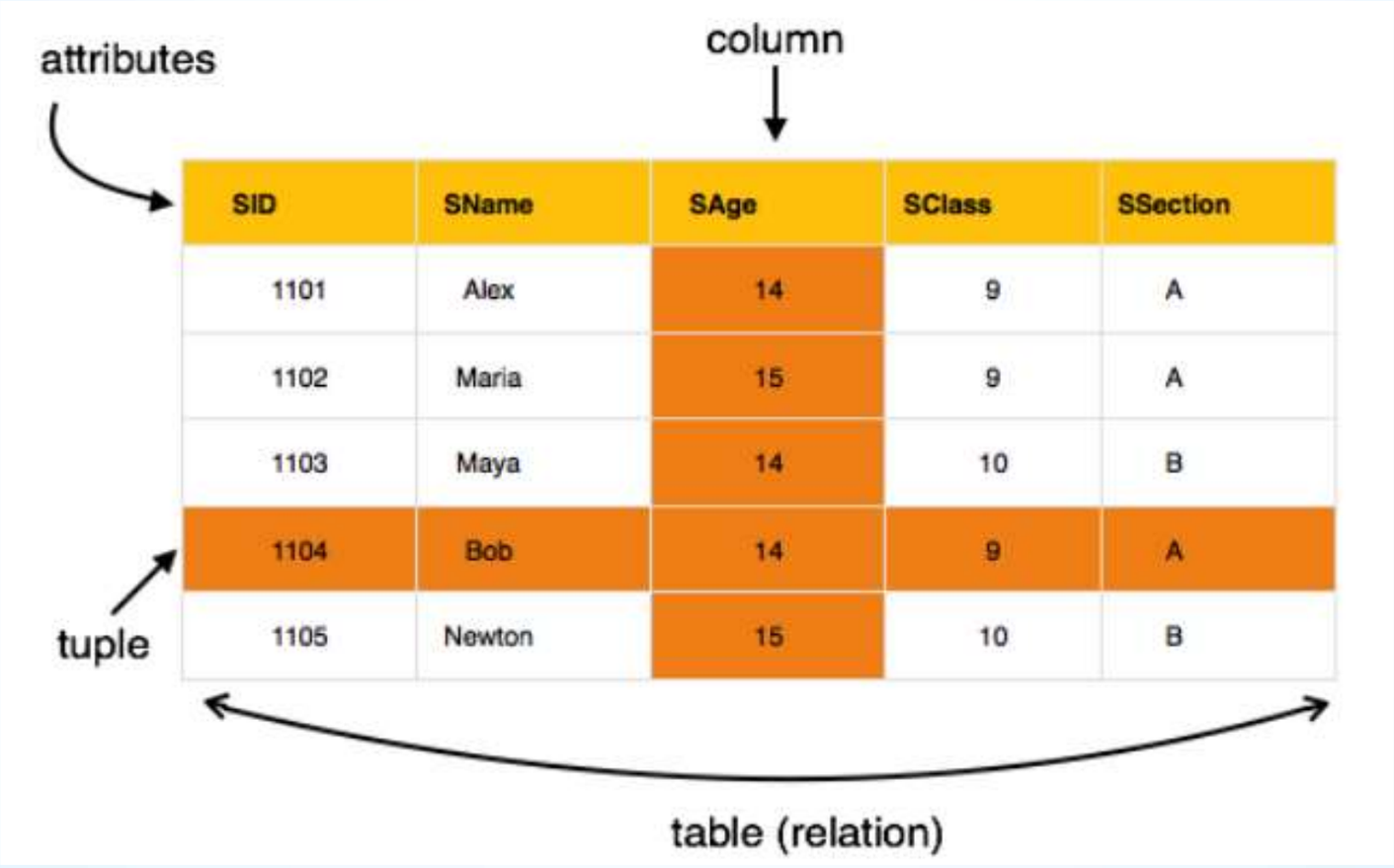
  Where each tuple is an ordered list of n values

  t = v1,v2,....., vn

  Where every vi belongs to domain (Ai) or includes null values.
- The relation schema is also known as 'intension' and relation state is also known as 'Extension'.

Relational Data Model

- Characteristic of the relational model is the usage of keys. These are specially designated columns within a relation, used to order data or relate data to other relations.



SID	SName	SAge	SClass	SSection
1101	Alex	14	9	A
1102	Maria	15	9	A
1103	Maya	14	10	B
1104	Bob	14	9	A
1105	Newton	15	10	B

Keys

- Relation key: Each row has one or more attributes, known as relation key, which can identify the row in the relation (table) uniquely.
- Primary key: One of the most important keys is the primary key, which is used to uniquely identify each row of data. A candidate key chosen as the principal means of identifying tuples within a relation. It cannot contain null values.
- Foreign key: A relation schema may have an attribute that corresponds to the primary key of another relation which is called as foreign key. Foreign keys relate data in one relation to the primary key of another relation.
  - E.g. customer_name and account_number attributes of depositor are foreign keys to customer and account resp.
  - Only values occurring in the primary key attribute of the referenced relation may occur in the foreign key attribute of the referencing relation.
- Alternate Key: A candidate key that is not the primary key is called alternate key.

Keys

- Every relation has some conditions that must hold for it to be a valid relation. These conditions are called Relational Integrity Constraints.
- These constraints are checked before performing any operation (insertion, deletion and updation) in database. If there is a violation in any of constrains, operation will fail.
- There are three main integrity constraints:
  - Key constraints
  - Domain constraints
  - Referential integrity constraints

Key Constraints

- There must be at least one minimal subset of attributes in the relation, which can identify a tuple uniquely. This minimal subset of attributes is called key for that relation.
- Key constraints are also referred to as Entity Constraints.
- Key constraints force that:
  - In a relation with a key attribute, no two tuples can have identical values for key attributes.
  - A key attribute cannot have NULL values.

Referential Integrity Constraints

- Referential integrity constraints work on the concept of Foreign Keys. A foreign key is a key attribute of a relation that can be referred in other relation.
- Referential integrity constraint states that if a relation refers to a key attribute of a different or same relation, then that key element must exist.
- Used to specify a relationship among tuples in two relations: the referencing relation and the referenced relation.
- The value in the foreign key column/columns (FK) of the referencing relation R1 can be either:
  - a value of an existing primary key value of a corresponding primary key PK in the referenced relation R2, or
  - a null.

Referential Integrity Constraints

STUDENT

ROLL_NO	NAME	ADDRESS	PHONE	AGE	BRANCH_CODE
1	RAM	DELHI	9455123451	18	CS
2	RAMESH	GURGAON	9652431543	18	CS
3	SUJIT	ROHTAK	9156253131	20	ECE
4	SURESH	DELHI		18	IT

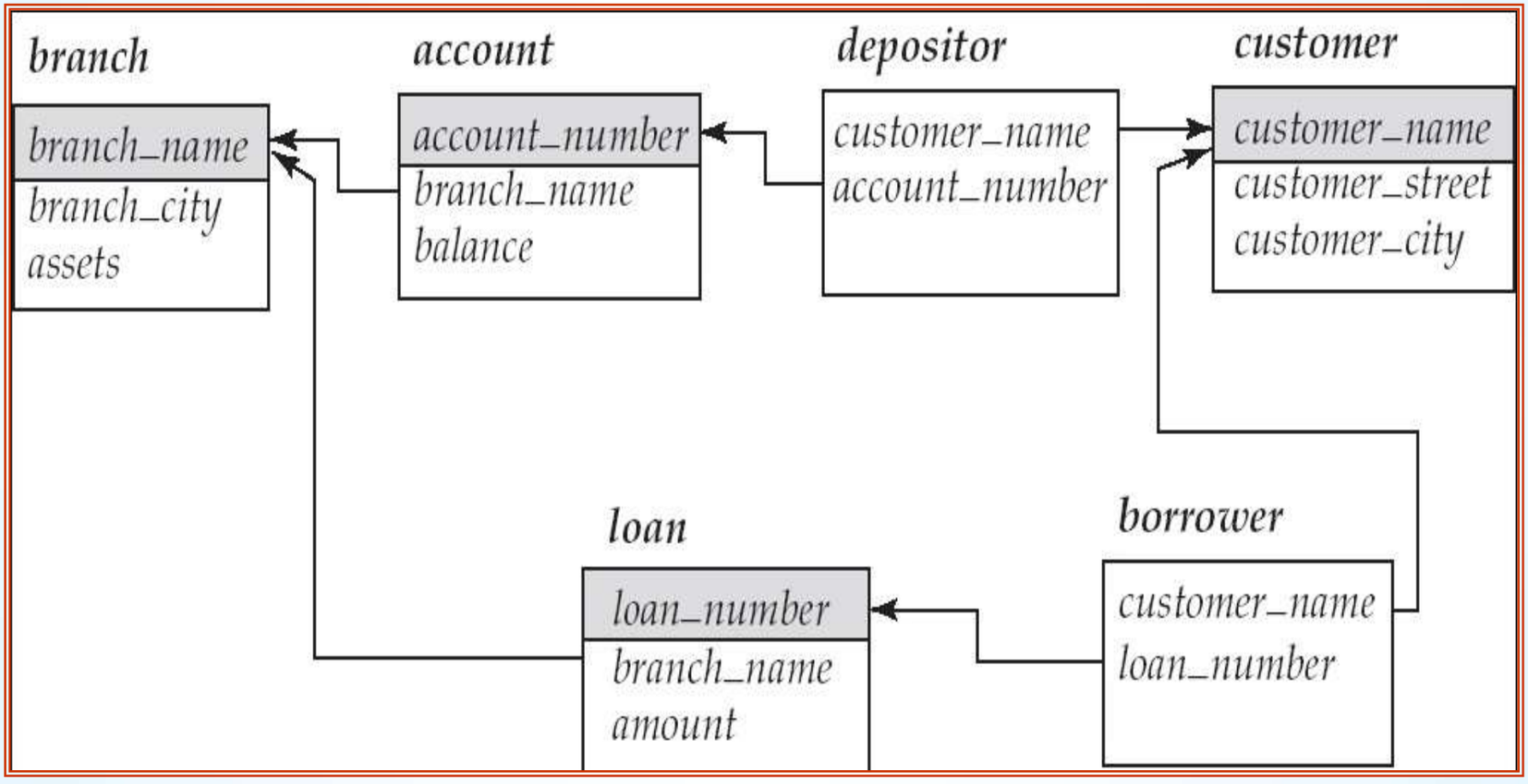
BRANCH

BRANCH_CODE	BRANCH_NAME
CS	COMPUTER SCIENCE
IT	INFORMATION TECHNOLOGY
ECE	ELECTRONICS AND COMMUNICATION ENGINEERING
CV	CIVIL ENGINEERING

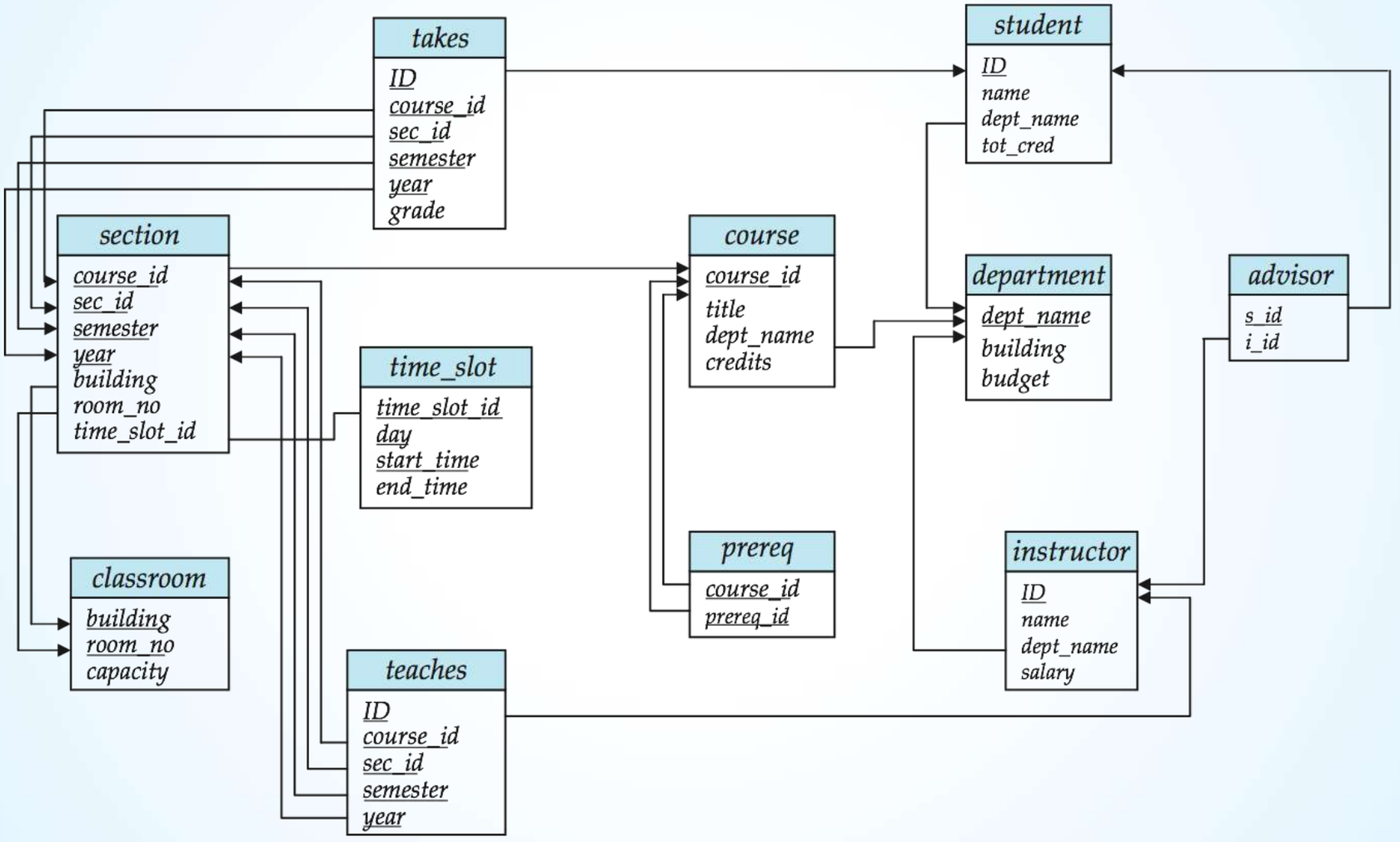
Domain Constraints

- Every value in a tuple must be from the domain of its attribute (or it could be null, if allowed for that attribute).
- An attribute can only take values which lie inside the domain range.
- Example: If a constraint AGE>0 is applied on STUDENT relation, inserting negative value of AGE will result in failure.

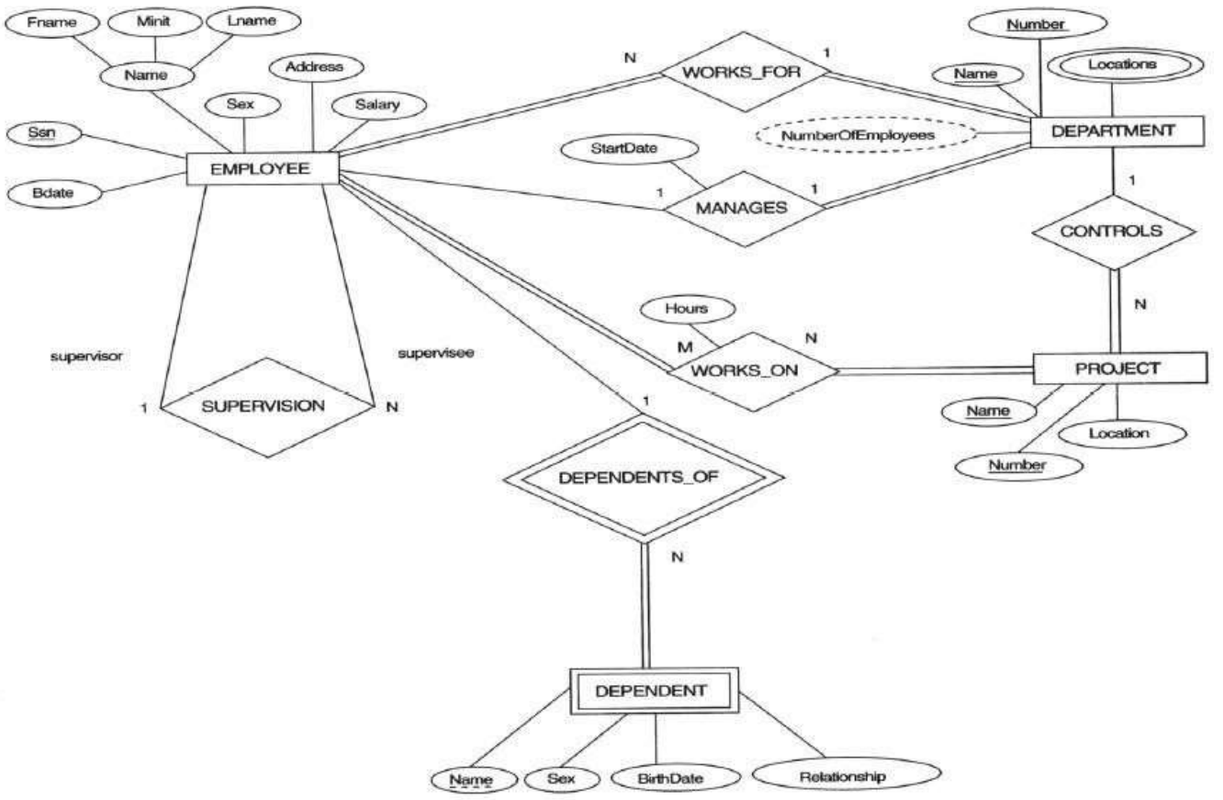
Schema Diagram



Schema Diagram



Conversion of EER Model to Relational Model



Step 1: Mapping of Regular Entity Sets.

• For each regular entity set E, create a relation R that includes all the simple attributes of E and include all the component attributes of composite attributes. Choose one of the key attributes of E as the primary key for R.

• *Example:* We create the relations EMPLOYEE, DEPARTMENT, and PROJECT in the relational schema. SSN, DNUMBER, and PNUMBER are the primary keys

Step 2: Mapping of Weak Entity Sets

• For each weak entity set W with owner entity set E, create a relation R and include all simple attributes and components of composite attributes of W as attributes of R. Also, include the primary key attribute of the E as foreign key attributes of R. The primary key of R is the *combination* of the primary key of the owner and the partial key of the weak entity set W.

• *Example:* Create the relation DEPENDENT. Include the primary key SSN of the EMPLOYEE relation as a foreign key attribute of DEPENDENT (renamed to ESSN). The primary key of the DEPENDENT relation is the combination {ESSN, DEPENDENT_NAME}

Step 3: Mapping of Binary 1:1 Relationship Sets

• For each binary 1:1 relationship set, let the entity sets be S and T that participate in Relation. Entity set with *total participation is S.* Include primary key of T as foreign key in S. Include any simple attributes and components of composite attributes of relationship set as attributes of S.

• *Example:* 1:1 relation MANAGES is mapped by choosing the participating entity set DEPARTMENT to serve in the role of S, because its participation in the MANAGES relationship set is total.

Step 4: Mapping of Binary 1:N Relationship Sets.

• For each regular binary 1:N relationship set, let the relation that represent the participating entity set at the N-side of the relationship set be S. Include the primary key of the relation T as foreign key in S. Include any simple attributes and components of composite attributes of the relationship Set as attributes of S.

• *Example:* For WORKS_FOR we include the primary key DNUMBER of the DEPARTMENT relation as foreign key in the EMPLOYEE relation and call it DNO.

Step 5: Mapping of Binary M:N Relationship Sets.

• For each regular binary M:N relationship set, create a new relation S to represent Relationship. Include the primary keys of the participating entity sets as foreign key attributes in S; their combination will form the primary key of S. Also include any simple attributes of the M:N relationship set and simple components of composite attributes as attributes of S.

• *Example:* The M:N relationship set WORKS_ON is mapped by creating a relation WORKS_ON in the relational schema. The primary keys of the PROJECT and EMPLOYEE relations are included as foreign keys in WORKS_ON and renamed PNO and ESSN. The primary key is {ESSN, PNO}. Attribute HOURS in WORKS_ON represents the HOURS attribute of the relation set.

Step 6: Mapping of Multivalued attributes.

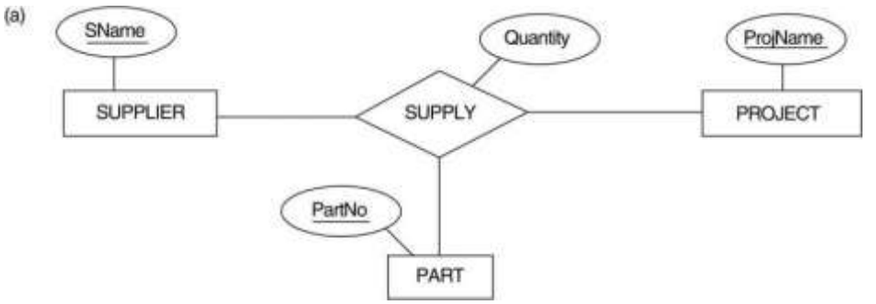
• For each Multivalued attribute A, create a new relation R. This relation R will include an attribute corresponding to A, plus the primary key attribute K as a foreign key in R. The primary key is the combination of A and K.

• *Example:* The relation DEPT_LOCATIONS is created. The attribute DLOCATION represents the multivalued attribute LOCATIONS of DEPARTMENT, while DNUMBER as foreign key-represents the primary key of the DEPARTMENT relation. The primary key of R is the combination of {DNUMBER, DLOCATION}.

Step 7: Mapping of N-ary Relationship Sets.

• For each n-ary relationship set R, where n>2, create a new relation S to represent R. Include the primary keys of the relations as foreign key attributes in S. Also include any simple attributes of relationship set and simple components of composite attributes as attributes of S.

• *Example:* The relationship set SUPPLY can be mapped to the relation SUPPLY, whose primary key is the combination of the three foreign keys {SNAME, PARTNO, PROJNAME}



Step8: Options for Mapping Specialization or Generalization

Option 8A: Multiple relations-Superclass and subclasses.

Create a relation L for C and relations Li for each subclass Si and PK(Li)=k. This option works for any specialization (total or partial, disjoint of over-lapping).

Option 8B: Multiple relations-Subclass relations only

Create a relation Li for each subclass Si and PK(Li) = k. This option only works for a specialization whose subclasses are total.

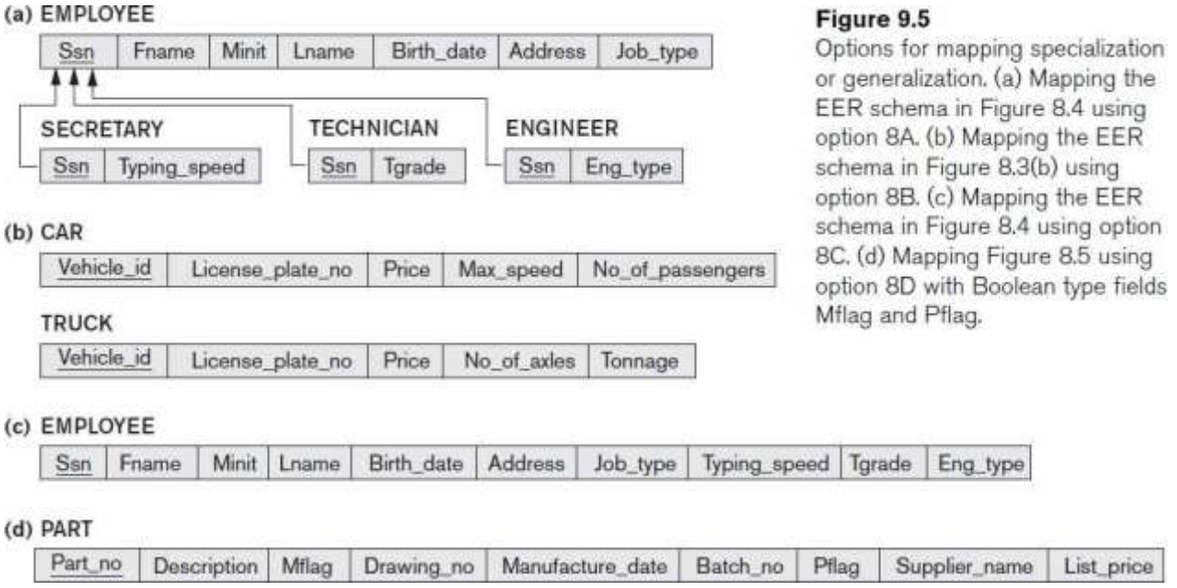


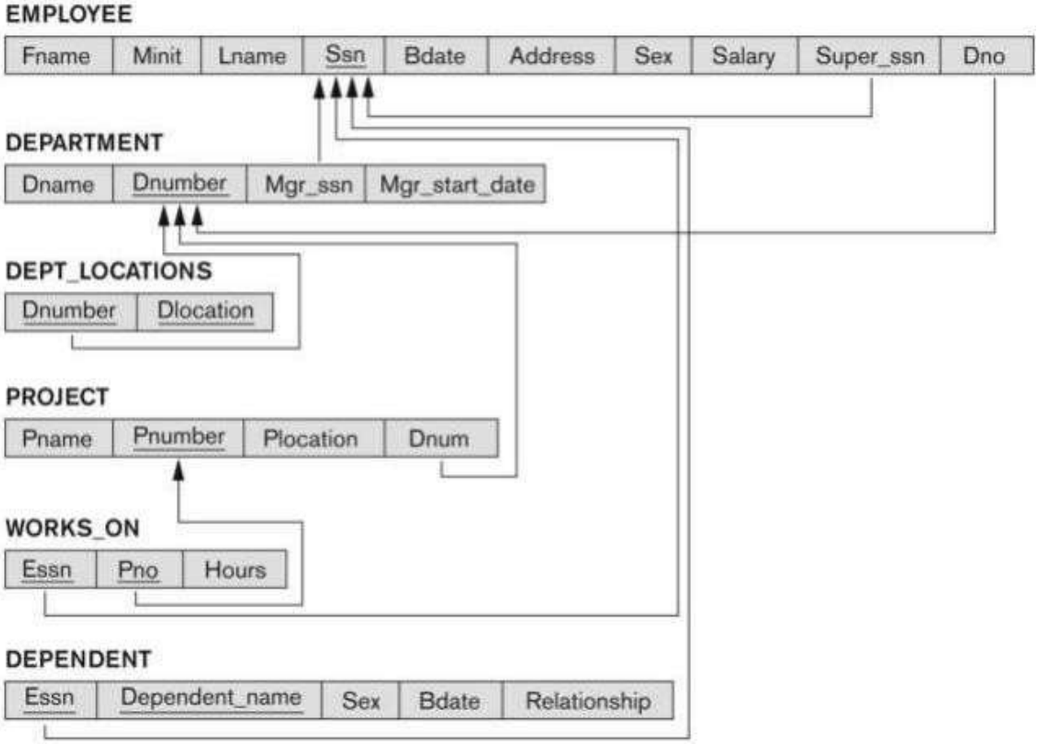
Figure 9.5
Options for mapping specialization or generalization. (a) Mapping the EER schema in Figure 8.4 using option 8A. (b) Mapping the EER schema in Figure 8.3(b) using option 8B. (c) Mapping the EER schema in Figure 8.4 using option 8C. (d) Mapping Figure 8.5 using option 8D with Boolean type fields Mflag and Pflag.

Option 8C: Single relation with one type attribute.

Create a single relation L with all the attributes including attributes of each subclass S and PK(L) = k.

Option 8D: Single relation with multiple type attributes.

Create a single relation L with all the attributes including attributes of each subclass S and PK(L) = k.
Each ti is a Boolean type attribute indicating whether a Tuple belongs to the subclass Si.



Relational Algebra is a procedural query language used to query the database tables to access data in different ways. It collects instances of relations as input and gives occurrences of relations as output. It uses various operations to perform this action.

Basic Relational Algebra Operations:

Select Operation (σ):

This is used to fetch tuples from relation who satisfies a given condition.

Syntax: $\sigma_p(r)$

Where σ stands for selection predicate and r stands for relation. p is prepositional logic formula which may use connectors like and, or, and not. These terms may use relational operators like: =, ≠, ≥, <, >, ≤.

Example:

$\sigma_{subject="database"}(Books)$

Output: Selects tuples from books where subject is 'database'.

$\sigma_{subject="database" \text{ and } price="450"}(Books)$

Output: Selects tuples from books where subject is 'database' and 'price' is 450.

$\sigma_{subject="database" \text{ and } price < "450" \text{ or } year > "2010"}(Books)$

Output: Selects tuples from books where subject is 'database' and 'price' is 450 or those books published after 2010.

Project Operation (π):

Project operation is used to project only a certain set of attributes of a relation.

Syntax: $\prod_{A1, A2...}(r)$

Example:

$\prod_{subject, author}$ (Books)

Output: Selects and projects columns named as subject and author from the relation Books.

$\prod_{Name, Age}(Student)$

Rename Operation (ρ):

This operation is used to rename the output relation for any query operation which returns result. It allows us to name the results of relational-algebra expressions. It also allows us to refer to a relation by more than one name.

Syntax: ρ(RelationNew, RelationOld)

Example: ρ(student1,students)

Syntax: $\rho_x(E)$ Where the result of expression E is saved with name of x.

Union Operation (∪):

This operation is used to fetch data from two relations. For this operation to work, the relations specified should have same number of attributes and compatible attribute domains. Also the duplicate tuples are automatically eliminated.

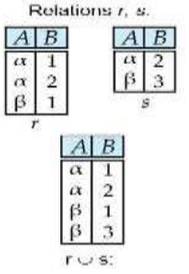
Syntax: A ∪ B

Example:

$\prod_{author}$ (Books) ∪ $\prod_{author}$ (Articles)

Output: Projects the names of the authors who have either written a book or an article or both.

$\prod_{Student}(RegularClass) \cup \prod_{Student}(ExtraClass)$



Intersection Operation (∩):

Defines a relation consisting of a set of all tuple that are in both A and B.For this operation to work, the relations specified should have same number of attributes and compatible attribute domains

Syntax: A ∩ B

$\prod_{author}(Books) \cap \prod_{author}(Articles)$

Output: Projects the names of the authors who have written both, a book and an article.

A	B
α	2

$r \cap s$

Set Difference (-):
The result of A - B, is a relation which includes all tuples that are in A but not in B. The attribute name of A has to match with the attribute name in B.
Syntax: A - B
Example: $\prod_{Student}(RegularClass) - \prod_{Student}(ExtraClass)$
Output: Projects Students who attend the regular class but not the extra class.

A	B
α	1
β	1

$r - s$

Cartesian Product (X):
This is used to combine data from two different relations into one and fetch data from the combined relation.
Syntax: A X B
Example: if we want to find the information for Regular Class and Extra Class which are conducted during morning, then, we can use the following operation:
$\sigma_{time = 'morning'}$ (RegularClass X ExtraClass)
For the above query to work, both RegularClass and ExtraClass should have the attribute time.

Relations r, s:

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

$r \times s$:

A	B	C	D	E
α	1	α	10	a
β	2	β	10	a
β	2	β	20	b

Example: $\sigma_{A=C}(r \times s)$

Natural Join ($\bowtie$):
This is used to combine data from two different relations into one and fetch data from the combined relation where only equality condition holds between common attributes. We can perform a Natural Join only if there is at least one common attribute that exists between two relations and that attributes must have the same name and domain. If t_r and t_s have the same value on each of the attributes in $R \cap S$, add a tuple t to the result.
$R = (A, B, C, D)$
$S = (E, B, D)$
Result schema = (A, B, C, D, E)
$r \bowtie s$ is defined as:
$\prod_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B = s.B \wedge r.D = s.D} (r \times s))$

Syntax: A $\bowtie$ B
Example: if we want to find the information for Regular Class and Extra Class which are conducted during morning, then, we can use the following operation:
$\sigma_{time = 'morning'}$ (RegularClass X ExtraClass)

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

s

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

$r \bowtie s$

Outer Join:
An inner join includes only those tuples with matching attributes and the rest are discarded in the resulting relation. Therefore, we need to use outer joins to include all the tuples from the participating relations in the resulting relation that avoids loss of information. It computes the join operation and then adds tuples from one relation that does not match tuples in the other relation to the result of the join. It uses null values which signifies that the value is unknown or does not exist. There are three kinds of outer joins: left outer join, right outer join and full outer join.

Left Outer Join (R ⟕ S):
All the tuples from the Left relation, R, are included in the resulting relation. If there are tuples in R without any matching tuple in the Right relation S, then the S-attributes of the resulting relation are made NULL. For example,

Relation Loan

loan_number	branch_name	amount
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

Relation Borrower

customer_nam	loan_number
Jones	L-170
Smith	L-230
Hayes	L-155

loan ⟕ borrower

loan_number	branch_name	amount	customer_name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	*null*

Right Outer Join: (R ⟖ S)
All the tuples from the Right relation, S, are included in the resulting relation. If there are tuples in S without any matching tuple in R, then the R-attributes of resulting relation are made NULL.

loan ⟖ borrower

loan_number	branch_name	amount	customer_name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-155	*null*	*null*	Hayes

Full Outer Join: (R ⟗ S)
All the tuples from both participating relations are included in the resulting relation. If there are no matching tuples for both relations, their respective unmatched attributes are made NULL.

loan ⟗ borrower

loan_number	branch_name	amount	customer_name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	*null*
L-155	*null*	*null*	Hayes

Banking Example

branch (branch_name, branch_city, assets)

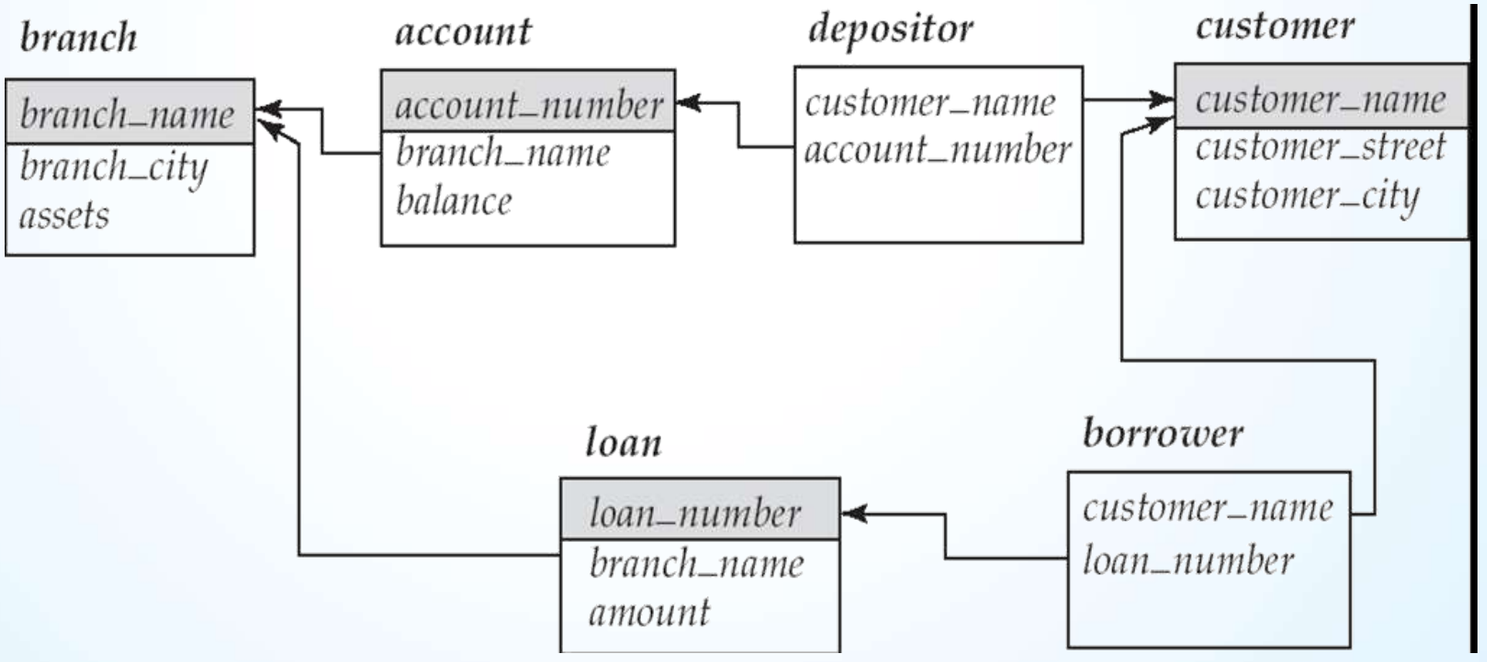
customer (customer_name, customer_street, customer_city)

account (account_number, branch_name, balance)

loan (loan_number, branch_name, amount)

depositor (customer_name, account_number)

borrower (customer_name, loan_number)



Example Queries

- Find all loans of over $1200

$$\sigma_{amount > 1200}(loan)$$

- Find the loan number for each loan of an amount greater than $1200

$$\Pi_{loan_number}(\sigma_{amount > 1200}(loan))$$

- Find the names of all customers who have a loan, an account, or both, from the bank

$$\Pi_{customer_name}(borrower) \cup \Pi_{customer_name}(depositor)$$

Bank Example Queries

- Find the names of all customers who have a loan and an account at bank.

$$\Pi_{customer_name}(borrower) \cap \Pi_{customer_name}(depositor)$$

- Find the name of all customers who have a loan at the bank and the loan amount

$$\Pi_{customer_name,\ loan_number,\ amount}(borrower \bowtie loan)$$

Example Queries

- Find the names of all customers who have a loan at the Perryridge branch.

$$\Pi_{customer_name}\ (\sigma_{branch_name=\text{“}Perryridge\text{”}}\ (\sigma_{borrower.loan_number\ =\ loan.loan_number}(borrower\ x\ loan)))$$

- Find the names of all customers who have a loan at the Perryridge branch but do not have an account at any branch of the bank.

$$\Pi_{customer_name}\ (\sigma_{branch_name\ =\ \text{“}Perryridge\text{”}}\ (\sigma_{borrower.loan_number\ =\ loan.loan_number}(\text{borrower x loan}))) - \Pi_{customer_name}(\text{depositor})$$

Bank Example Queries

- Find the largest account balance

$$\Pi_{balance}(account) - \Pi_{account.balance}$$
$$(\sigma_{account.balance < d.balance} (account \times \rho_{d} (account)))$$

Bank Example Queries

- Find all customers who have an account from at least the “Downtown” and the Uptown” branches.
  - Query 1

$$\Pi_{customer_name}(\sigma_{branch_name = \text{“Downtown”}}(depositor \bowtie account)) \cap$$

$$\Pi_{customer_name}(\sigma_{branch_name = \text{“Uptown”}}(depositor \bowtie account))$$